Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

LAB 08	
Aim:	Apply the concept of generic function and generic class in java.
Theory:	Generics means parameterized types. The idea is to allow a type (like Integer, String, etc., or user-defined types) to be a parameter to methods, classes, and interfaces. Generics in Java allow us to create classes, interfaces, and methods where the type of the data is specified as a parameter. If we use generics, we do not need to write multiple versions of the same code for different data types. Why Use Generics? Before Generics, the Java developers used Object to store any type of data. The Object is the superclass of all other classes, and an Object reference can refer to any object. These features lack type safety. For example, adding an Integer to a list of String would not show an error until runtime. Generics add that type of safety feature. We will discuss that type of safety feature in later examples. Generics in Java are similar to templates in C++. For example, classes like HashSet, ArrayList, HashMap, etc., use generics very well. There are some fundamental differences between the two approaches to generic types. Types of Java Generics 1. Generic Method: A generic Java method takes a parameter and returns some value after performing a task. It is exactly like a normal function; however, a generic method has type parameters that are cited by an actual type. This allows the generic method to be used in a more general way. The compiler takes care of the type of safety, which enables programmers to code easily since they do not have to perform long, individual type castings. 2. Generic Classes: A generic class is implemented exactly like a non-generic class. The only difference is that it contains a type parameter section. There can be more than one type of parameter, separated by a comma. The classes that accept one or more parameters are known as parameterized classes or parameterized types.

Program:	// sheikh wasimuddin 23071044 cseiot-73
	public class exp_8 {
	  public static <T> void display(T value) {
	  System.out.println("Value: " + value);
	  }
	  static class Box<T> {
	  private T item;
	  public void set(T item) {
	  this.item = item;
	  }
	  public T get() {
	  return item;

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
        }
    }
    public static void main(String[] args) {
        display("sheikh wasimuddin");
        display(73);
        Box<String> stringBox = new Box<>();
        stringBox.set("Generic Class Example");
        System.out.println("Box contains: " + stringBox.get());
    }
}
```

Output:

exp_8.java

```
public class exp_8 {
    static class Box<T> {

        public void set(T item) {
            this.item = item;
        }

        public T get() {
            return item;
        }
    }

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        // Using generic method
        display(value:"sheikh wasimuddin");
        display(value:73);

        // Using generic class
        Box<String> stringBox = new Box<>();
        stringBox.set(item:"Generic Class Example");
        System.out.println("Box contains: " + stringBox.get());
    }
}
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\student\Desktop\Iot-73>  & 'C:\Program Files\Java\jdk-22\bin\java.exe' '-XX:
edhat.java\jdt_ws\Iot-73_908e7a1\bin' 'exp_8'
Value: sheikh wasimuddin
Value: 73
Box contains: Generic Class Example
PS C:\Users\student\Desktop\Iot-73>
```

Conclusion

Implementing the concept generic function and generic class in java executed successfully.

Name Of Faculty:

Prof. Akhil Jajulwar

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)